A PROJECT REPORT
on

QLens: From a Messy Spreadsheet to a Verified Third-Normal-Form Database

Submitted to Manipal University Jaipur
Towards the partial fulfillment for the Award of the Degree of

B. Tech Computer Science and Engineering

2026–2027

By

Aditya Gaur
Reg. No. 2502051500

Aditya Choudhary
Reg. No. 2502051502

Semester III, Section K3 · Relational Database Management Systems

[University logo: figures/muj-logo.png, 4.56 in × 0.9 in]

Under the guidance of

Ms. Surabhi Agrawal

Department of Artificial Intelligence and Machine Learning
Manipal University Jaipur

Jaipur, Rajasthan

Abstract

Small offices keep their records in a single, ever-growing spreadsheet. A coaching institute's fee register repeats every teacher's phone number on hundreds of rows, spells the same student three different ways, and needs an hour of filtering to answer "who has not paid for October?". These are the update, insertion and deletion anomalies that normalization theory was invented to remove, but the people who suffer from them have never heard the terms and the tools they use do not help: database front-ends expect the user to design the tables, and chat-style assistants answer one question without leaving a correct database behind.

QLens is a local web application that takes such a sheet and produces a normalized PostgreSQL database from the data itself. The file is copied unchanged into a raw table and a typed staging table, with column types inferred by SQL. Candidate keys and functional dependencies are then discovered inside the database by PL/pgSQL functions (`qlens.check_fd`, `qlens.is_unique`) that group the staged rows with dynamic SQL; a dependency that holds on all but a few rows is reported as a data inconsistency together with the rows that break it. Near-duplicate spellings are found with `pg_trgm` trigram similarity over a GIN index, and every repair is logged. From the accepted exact dependencies QLens computes a minimal cover and runs Bernstein's 3NF synthesis, extended with a key relation so that the decomposition is always lossless, and turns the result into tables with surrogate keys, `UNIQUE NULLS NOT DISTINCT` natural keys and foreign keys. The design is applied in a single transaction and accepted only if a view that joins the new tables back together returns exactly the staged rows, compared as multisets with `EXCEPT ALL` in both directions.

QLens keeps its own metadata in a normalized `qlens` schema of 22 tables with CHECK constraints, generated columns, triggers and views that use window functions. Each workspace has its own schemas and its own low-privilege PostgreSQL login role, so user queries are isolated by privileges rather than by convention. On the bundled 3,000-row fee register the analysis finds the two composite candidate keys, six core dependencies and the 13 planted faulty rows, and proposes the textbook four-table design (`students`, `batches`, `teachers`, `fee_register`); applying it rebuilds all 3,000 rows exactly. The algorithms are verified by unit tests on textbook examples, by 900 randomized synthesis cases checked against a chase-based lossless-join oracle and a brute-force 3NF test, and by end-to-end tests that apply the generated SQL to randomly generated sheets (229 backend and 63 frontend tests pass). The design canvas, the sandboxed query editor and a tool-using assistant are in progress, and every capability is marked with its build status.

Keywords: functional dependency discovery, third normal form, Bernstein synthesis, lossless-join decomposition, PostgreSQL, PL/pgSQL, role-based isolation, data cleaning, `pg_trgm`.

Table of Contents

Abstract i

Table of Contents ii

List of Figures iii

List of Tables iv

1 Introduction 1

1.1 Scope of the Work 2

1.2 Product Scenarios 3

2 Requirement Analysis 4

2.1 Functional Requirements 4

2.2 Non-functional Requirements 4

2.3 Use Case Scenarios 6

2.4 Other Software Engineering Methodologies 6

3 System Design 9

3.1 Design Goals 9

3.2 System Architecture 9

3.3 Detailed Design Methodologies 11

4 Conclusion 21

References 22

A Sample Datasets and Screens 23

List of Figures

Figure 1.1 Landing page 2
Figure 1.2 Workspace dashboard 3

Figure 2.1 Import screen 5
Figure 2.2 Use-case diagram 7

Figure 3.1 Layered architecture 10
Figure 3.2 ER diagram of the metadata schema 12
Figure 3.3 Normalization pipeline 14
Figure 3.4 Fee-register example 17
Figure 3.5 Query sequence 19
Figure 3.6 Analyze screen 20

Figure A.1 Screen gallery 24

List of Tables

Table 2.1 Functional requirements 4
Table 2.2 Non-functional requirements 6
Table 2.3 Use cases 8
Table 2.4 Test summary 8

Table 3.1 Technology stack 10
Table 3.2 Integrity rules 11

Table A.1 Sample datasets 23

CHAPTER 1

Introduction

Small offices run on one ever-growing spreadsheet. A coaching institute's fee register is typical: one row per student per month, with the student's name and phone, the batch, the batch teacher and the teacher's phone, the month, the fee paid and the payment mode. Over a year it reaches thousands of rows and develops the classic anomalies of an unnormalized relation [1, 2]:

- **Update anomaly.** A teacher's phone number is repeated on every row of every student of that teacher; when it changes, a few old rows are always missed, and the sheet now holds two numbers.
- **Inconsistency.** "Riya Sharma" and "riya sharma" are counted as two students, so totals and dues are wrong.
- **Insertion and deletion anomalies.** A new teacher cannot be recorded until a student pays; deleting the last row of a batch deletes the only record of who teaches it.
- **Query cost.** "Who has not paid for October?" takes an hour of filtering.

In database terms the sheet violates third normal form: `teacher_phone` depends on `teacher`, which depends on `batch`, which depends on the student, which is only part of the key (`student_name`, `month`). The people who suffer from this do not know the vocabulary, and the tools they use do not help: database front-ends such as Airtable or NocoDB expect the user to design the tables, and chat assistants answer one question about a file without leaving a correct database behind.

QLens closes this gap. It reads the sheet into PostgreSQL, discovers from the data itself which columns determine which (functional dependencies) and which columns identify a row (candidate keys), shows the rows that break an almost-true rule as inconsistencies, finds near-duplicate spellings, and then synthesizes a third-normal-form design [3]. With one click the design is created as real tables with primary, unique and foreign keys, the data is migrated in a single transaction, and the migration is accepted only if joining the new tables back gives exactly the original rows. The objectives were therefore:

O1. Do the analysis *inside* the database, with SQL and PL/pgSQL, so that its correctness rests on the same engine that will hold the data.

O2. Produce designs that are provably in 3NF and lossless, and verify losslessness on the real rows.

O3. Isolate every user and workspace by PostgreSQL privileges, not by application convention.

O4. Present every result in plain language, for users who have never written SQL.

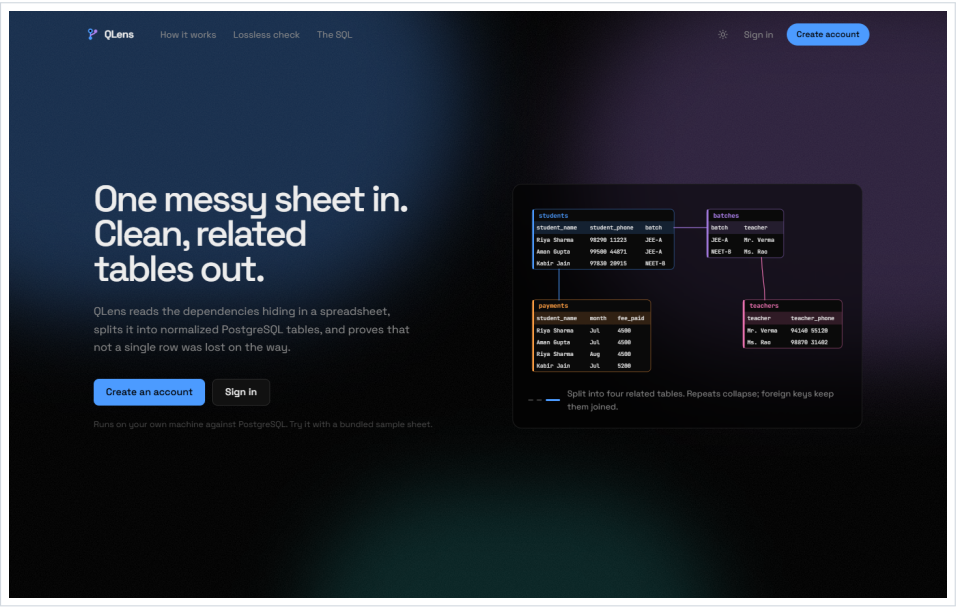


Figure 1.1: The QLens landing page: one messy sheet in, clean related tables out.

1.1 Scope of the Work

In scope. Import of one CSV or XLSX sheet per workspace (up to 20 MB, 100,000 rows and 60 columns); SQL type inference and profiling; discovery of candidate keys of up to three columns and of exact and approximate functional dependencies with a left-hand side of up to two columns; near-duplicate detection and repair with an audit log; minimal cover and 3NF synthesis; generation of the DDL; a transactional apply with a lossless check; a sandboxed, read-only query editor with history; and three bundled sample sheets with planted errors (a coaching fee register, a kirana shop's sales and a hostel mess register) so that the product can be demonstrated without private data. QLens keeps its own records in a normalized metadata schema, which is itself a main subject of this report.

Out of scope. Other database engines, multi-sheet workbooks, live collaboration, writes from the query editor, normal forms beyond 3NF (BCNF can lose dependencies; 3NF synthesis cannot), and the hardening needed for public multi-tenant hosting. QLens runs locally on one laptop against a private PostgreSQL 18 cluster.

Status. The report describes the system at the time of submission and marks each capability as Built (implemented and covered by automated tests: 229 backend tests and 63 frontend tests pass), In progress (partly implemented) or Planned (designed, not yet started). Nothing is described as working unless it is marked Built.

1.2 Product Scenarios

The coaching-institute owner. Built The owner imports the fee register (3,000 rows: 300 students, 8 batches, 6 teachers, 10 months). QLens reports the keys (`student_name`, `month`) and (`student_phone`, `month`), six core dependencies, and two inconsistencies: four students spelled differently on 8 rows, and Mr. Gupta's old phone number on 5 rows. After "accept all suggestions" it proposes `students`, `batches`, `teachers` and `fee_register`; applying the design rebuilds all 3,000 rows exactly. A teacher's phone is now stored once.

The developer migrating legacy data. Built A developer uploads an undocumented CSV export and reads off the keys and dependencies that actually hold, with the share of violating rows next to each approximate one, which shows where the legacy data cannot be trusted. The proposal comes with generated `CREATE TABLE` statements that can be copied into the application's migrations, or QLens can migrate the data itself and prove that nothing was lost.

The student learning normalization. Built for analysis and design, In progress for querying. A student opens the hostel mess sample and sees why `room` determines `hostel_block`, how the minimal cover is derived and why each table of the proposal exists, then queries the result. A clause-by-clause visualizer and a performance laboratory with real execution plans are Planned.

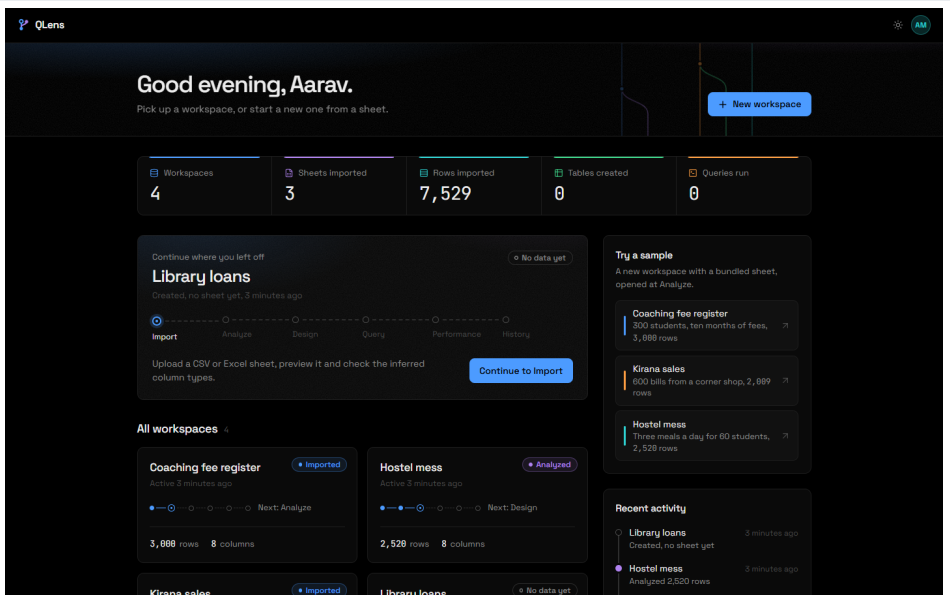


Figure 1.2: The workspace dashboard (dark theme): statistics, the workspace to continue, and one-click sample sheets.

CHAPTER 2

Requirement Analysis

The requirements come from the anomalies of Chapter 1, from the three target users and from the three sample sheets, whose planted errors double as acceptance tests.

2.1 Functional Requirements

Table 2.1: Functional requirements and their implementation status.

ID	Requirement	Status
FR-1	**Accounts and workspaces.** Sign-up and sign-in; each workspace gets its own schemas and database role.	Built
FR-2	**Import.** CSV (encoding and delimiter detected) or XLSX; raw cells kept unchanged in `_raw`.	Built
FR-3	**Type inference and profile.** Infer integer, numeric, boolean, date (incl. day-first) or text in SQL; null and distinct counts.	Built
FR-4	**Key and FD discovery.** Candidate keys (1–3 columns), exact and approximate dependencies (LHS 1–2), computed in PostgreSQL.	Built
FR-5	**Clean-up.** Near-duplicate groups via trigram similarity; conflicting rows of approximate dependencies with a suggested value; every edit logged.	Built
FR-6	**3NF proposal.** Minimal cover, Bernstein synthesis, surrogate and foreign keys, names, explanations, DDL; renames while a draft.	Built
FR-7	**Apply.** Create and load the tables in one transaction; commit only if the join of the new tables equals the cleaned rows.	Built
FR-8	**Design canvas.** Draggable ER diagram with rename, SQL panel and apply.	In progress
FR-9	**Sandboxed queries and history.** Read-only SELECTs as the workspace role, timeout and row cap; saved queries and run log.	In progress
FR-10	**Assistant.** Optional Gemini command bar: answers questions, edits the draft only through validated tools.	In progress
FR-11	**Performance lab and step-through.** Plan tree from `EXPLAIN ANALYZE`, index experiments, clause-by-clause row counts.	Planned
FR-12	**Samples.** Three generated sheets with known, planted problems.	Built

2.2 Non-functional Requirements

A text filter such as “the statement must start with `SELECT`” is easily bypassed with a comment, a CTE or a function that writes. NFR-1 is therefore met by layers that fail independently: privileges, a read-only transaction, a timeout and a row cap, all enforced by PostgreSQL itself [4].

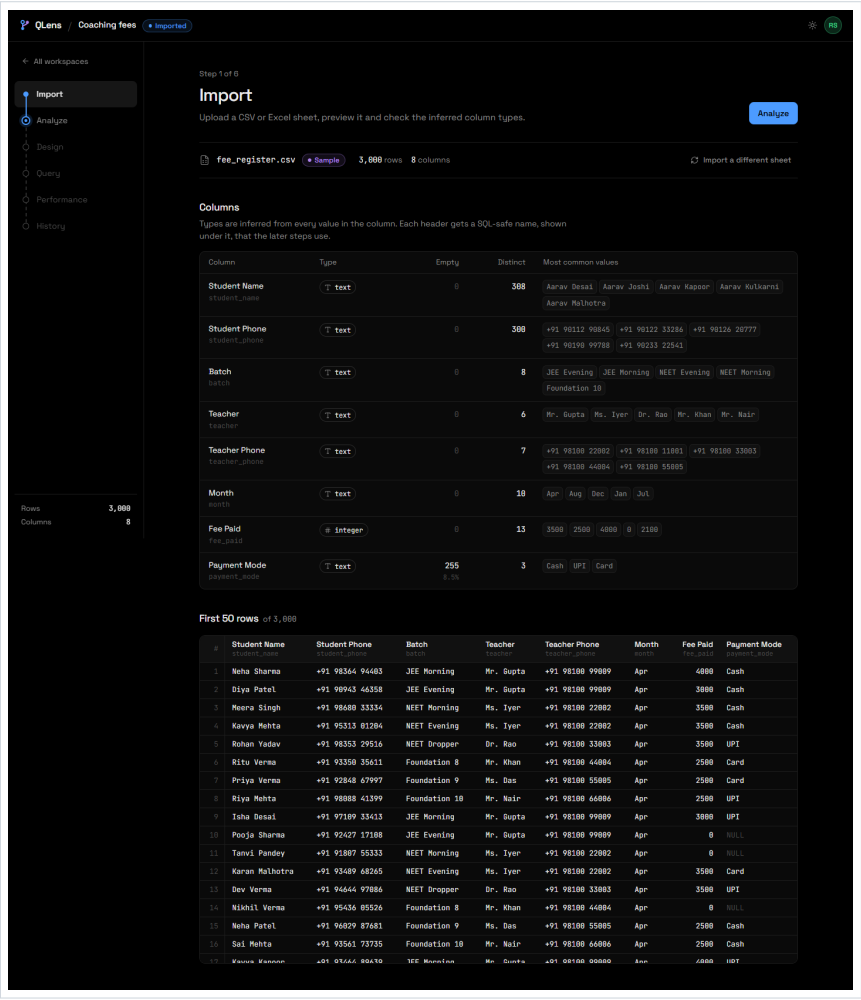


Figure 2.1: Import (FR-2, FR-3): inferred SQL types, null and distinct counts, frequent values and a preview of the staged rows.

Table 2.2: Non-functional requirements and the mechanism that meets each.

ID	Quality	Mechanism
NFR-1	Isolation	One PostgreSQL `LOGIN` role per workspace with privileges only on its schemas; read-only sessions, 5 s `statement_timeout`, 1,000-row cap.
NFR-2	Correctness	3NF and losslessness by construction, property-tested; `EXCEPT ALL` check on the real rows at apply time.
NFR-3	Integrity	The database rejects bad metadata: `CHECK`, `UNIQUE`, foreign keys, generated columns, partial unique indexes.
NFR-4	Atomicity	Import, every clean-up and apply each run in one transaction; failure leaves no partial state.
NFR-5	Performance	Analysis of the 3,000-row sample runs inside PostgreSQL within the 5 s bound asserted by a test; trigram search uses a GIN index.
NFR-6	Auditability	`_raw` is never edited; each fix is logged in `dataset_fixes` with old value, new value and row count.
NFR-7	Security	Argon2id password hashes; only SHA-256 digests of session tokens stored; every identifier quoted, every value bound.
NFR-8	Usability	Plain-English findings, light and pure-black themes, motion that respects reduced-motion settings.

2.3 Use Case Scenarios

Detailed flow of UC-5. *Precondition:* a draft proposal exists for an analyzed dataset. *Main flow:* the user presses Apply; the server locks the draft, drops and recreates the workspace's main schema, creates and loads the tables, builds the `original_sheet` view and runs the lossless check; on success the proposal and dataset become *applied* and the tables are analyzed. *Alternative:* if the check finds a difference, the server answers 409 with up to five differing rows and the transaction is rolled back, so the previous tables and statuses are untouched.

2.4 Other Software Engineering Methodologies

Iterative, test-driven delivery. The system was built in thin vertical increments in the order of value: database foundation and accounts, then import, discovery, clean-up, synthesis, proposal and apply, then querying. Each increment started from failing tests [5] and was followed by a review whose findings became regression tests; for example, a randomized test showed that plain synthesis without a key relation was lossy in 74 of 3,000 random cases, which led to the key step of Algorithm 3.1. The schema is managed by numbered SQL migrations applied by a small runner, and the test database is rebuilt from them at the start of every session.

Test results. Table 2.4 summarizes the suites. Database tests run against a real PostgreSQL 18 instance, because the central logic lives in SQL and cannot be meaningfully mocked.



Figure 2.2: Use-case diagram. The three target groups specialise a signed-in user and inherit the common flow (import, analyse, clean, design, apply, query); each group adds the use cases it motivates. Colours follow the pipeline phases of Figure 3.3.

Table 2.3: Principal use cases.

UC	Goal	Main flow and outcome	Status
UC-1	Import a sheet	Upload or pick a sample; raw and typed copies staged; columns profiled.	Built
UC-2	Review data health	Keys and dependencies in plain English; reject a coincidental one.	Built
UC-3	Clean the data	Merge spellings, accept suggested values or all; re-analysis follows.	Built
UC-4	Generate a design	3NF proposal with explanations and SQL; rename tables and columns.	Built
UC-5	Apply the design	One transaction; lossless check passes or everything rolls back.	Built
UC-6	Query the database	Read-only SQL as the workspace role; run recorded in history.	In progress
UC-7	Ask the assistant	Questions answered; draft edited through validated tools, with undo.	In progress
UC-8	Study a query	Plan tree, index experiment, step-through.	Planned

Table 2.4: Automated tests (latest full run: 229 backend cases and 63 frontend tests, all passing).

Area	Cases	What is verified
Schema, migrations, auth, workspaces	39	constraints, generated columns, triggers, views; role and schema set-up
Import (parse, stage, routes, samples)	39	encodings, delimiters, header names, type inference, planted errors
Discovery and clean-up	53	`check_fd`, keys, noise rules, trigram groups, fixes, rollback
FD algebra	19	closure, minimal cover, synthesis; 900 random cases checked for 3NF and losslessness
Naming, proposal and apply	79	FK substitution, renames, generated SQL on random sheets, apply, rollback, catalog
Frontend (Vitest)	63	API client, formatting, lane colours, dashboard and analysis view models

CHAPTER 3

System Design

3.1 Design Goals

G1. **Correct by construction, verified on data.** Designs are 3NF and lossless by the algorithm, and every apply re-checks losslessness on the real rows before committing.

G2. **SQL in plain sight.** No ORM: constraints, PL/pgSQL, triggers, views, dynamic SQL and catalog queries are hand-written, and the generated SQL is shown to the user.

G3. **Work where the data is.** Profiling, dependency checks and duplicate search run inside PostgreSQL; only the set algebra of synthesis runs in Python.

G4. **Isolation by privilege.** A user's query can reach only its own workspace because the database role lacks the privileges, not because the application filters text.

G5. **Non-destructive and auditable.** The raw import is never modified; every repair is a logged, transactional edit of a separate staging copy.

3.2 System Architecture

QLens is a three-tier web application whose bottom tier does unusually much work (Figure 3.1). The *client* is a React single-page application [6]; it holds no authoritative state and talks JSON to `/api/*` with the session in an `httpOnly` cookie. The *application tier* is FastAPI on Python 3.13 [7] with psycopg 3 [8]; each request runs in one transaction on a pooled connection as the owning role `qlens_app`, committed before the response is sent so that a commit failure can never follow a success reply. The *data tier* is a private PostgreSQL 18 cluster [4] with the `qlens` metadata schema and, per workspace, three schemas and one role:

- `ws_<id>_src`: `_raw` (every cell as text, never edited) and `_staging` (typed, the only table clean-up edits);
- `ws_<id>`: the generated 3NF tables and the `original_sheet` view;
- `ws_<id>_lab`: a scratch copy for performance experiments (Planned);
- `qlens_ws_<id>`: a `LOGIN`, `NOINHERIT` role with `CONNECTION LIMIT 3`, role-level settings that make every session read-only, stop statements after 5 s and end idle transactions after 10 s, and `USAGE`/`SELECT` on its own schemas only (with `ALTER DEFAULT PRIVILEGES` for future tables). Its password is an HMAC of the workspace id under the server secret, so it is never stored. Role and schemas are created in the same transaction as the workspace row and dropped with it.

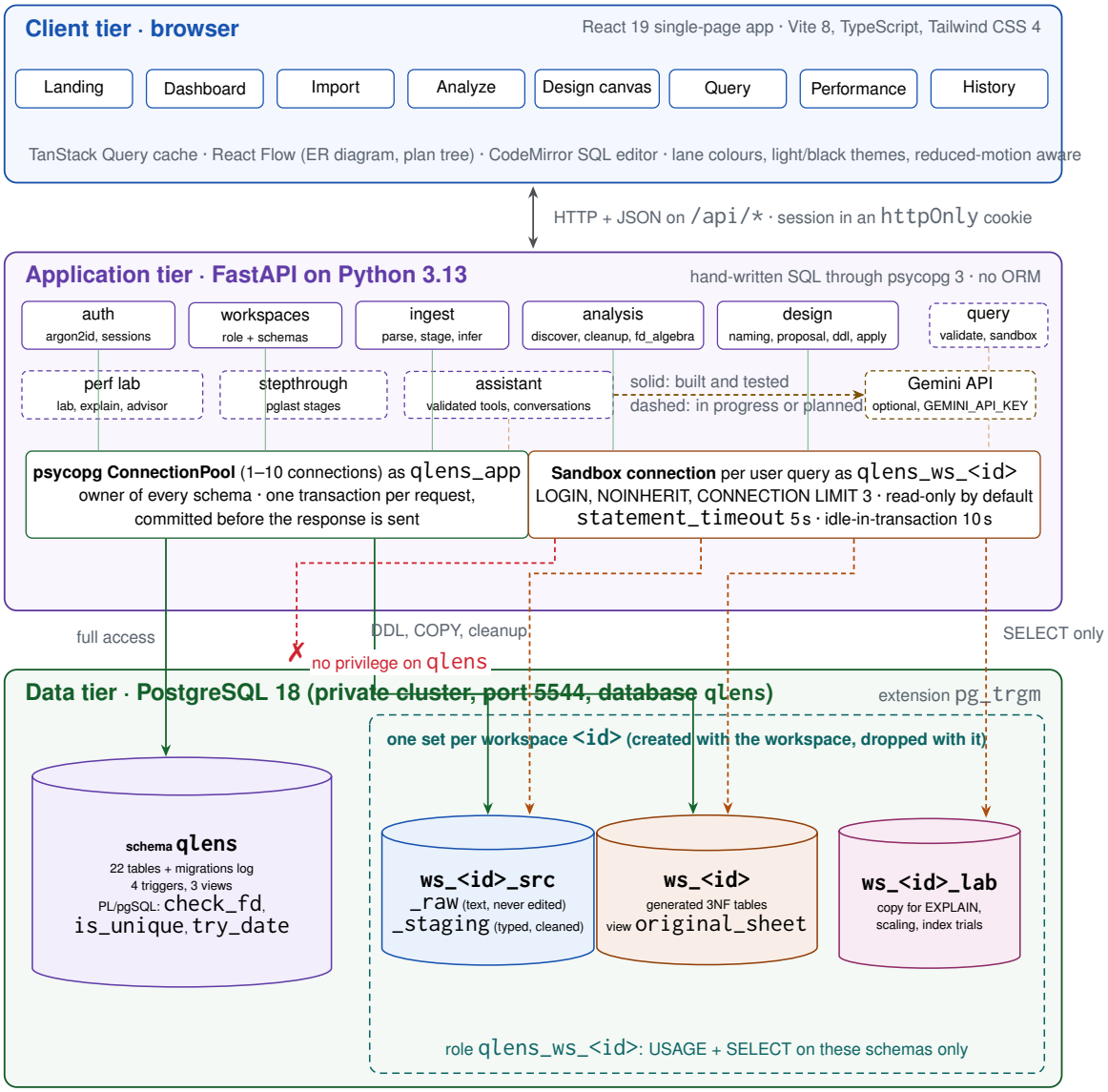


Figure 3.1: Layered architecture of QLens. Application code reaches PostgreSQL through two kinds of connection: the shared pool as the owning role, and a per-query sandbox connection as the workspace's own low-privilege LOGIN role.

Table 3.1: Technology stack.

Layer	Technology
Database	PostgreSQL 18 (private cluster), PL/pgSQL, `pg_trgm` [9]
Backend	Python 3.13, FastAPI, psycopg 3 with connection pool, pglast [10], openpyxl, argon2-cffi
Frontend	React 19, TypeScript, Vite, Tailwind CSS, TanStack Query, React Flow, CodeMirror
Testing	pytest against a real PostgreSQL test database; Vitest

3.3 Detailed Design Methodologies

3.3.1 Metadata Schema

QLens's own data is a normalized design of 22 tables plus a migrations log (Figure 3.2). Users own workspaces; a workspace holds at most one dataset (`UNIQUE workspace_id`); a dataset has columns, candidate keys, functional dependencies, duplicate groups and a log of fixes; designs are `schema_proposals` with their `proposed_tables`, `proposed_columns` and unique keys; and the query side records saved queries, runs, plans and index experiments. Multi-column keys and dependency left-hand sides are junction tables (`candidate_key_columns`, `fd_lhs_columns`), so no column stores a list of column names. The only redundancy is deliberate and documented: for example `dataset_id` is kept next to `column_id` for cheap filtering and cascades.

Table 3.2: Selected integrity rules enforced by the metadata schema.

Rule	Purpose
`CHECK (status <> 'applied' OR lossless_ok IS TRUE)`	a design cannot be recorded as applied without a passed check (`IS TRUE`: NULL must fail)
partial `UNIQUE (dataset_id) WHERE status = 'draft'` and `. . . = 'applied'`	at most one draft and one applied design per dataset
`is_exact`, `violation_ratio`, `speedup GENERATED . . . STORED`	derived values can never disagree with their inputs
`CHECK ((role = 'fk') = (references_table_id IS NOT NULL))`	a column is a foreign key exactly when it references a table
`CHECK (name ~ '^[a-z_][a-z0-9_]{0,62}$')`	every stored name is a safe identifier within 63 bytes
`ON DELETE CASCADE` / `SET NULL`	no orphans; run history survives deleting a saved query
index `query_runs (workspace_id, created_at DESC)`	serves the history page and the summary view (foreign keys are not indexed automatically, so each FK has a covering index)

Triggers and views. `touch_updated_at` stamps `updated_at` with `clock_timestamp()` on every update. `record_query_run` (Listing 3.1) keeps a saved query's counters in step with its successful runs; its `WHEN` clause filters before the function is called. `v_workspace_summary` feeds the dashboard through two `LEFT JOIN LATERAL` aggregates, `v_slow_queries` uses `percentile_cont(0.9) WITHIN GROUP`, and `v_saved_query_trend` uses the window functions `row_number`, `lag` and a five-run moving `avg`.

Listing 3.1: AFTER INSERT trigger maintaining saved-query statistics (`002_logic.sql`)

```
1 CREATE FUNCTION qlens.record_query_run() RETURNS trigger LANGUAGE plpgsql AS $$
2 BEGIN
```

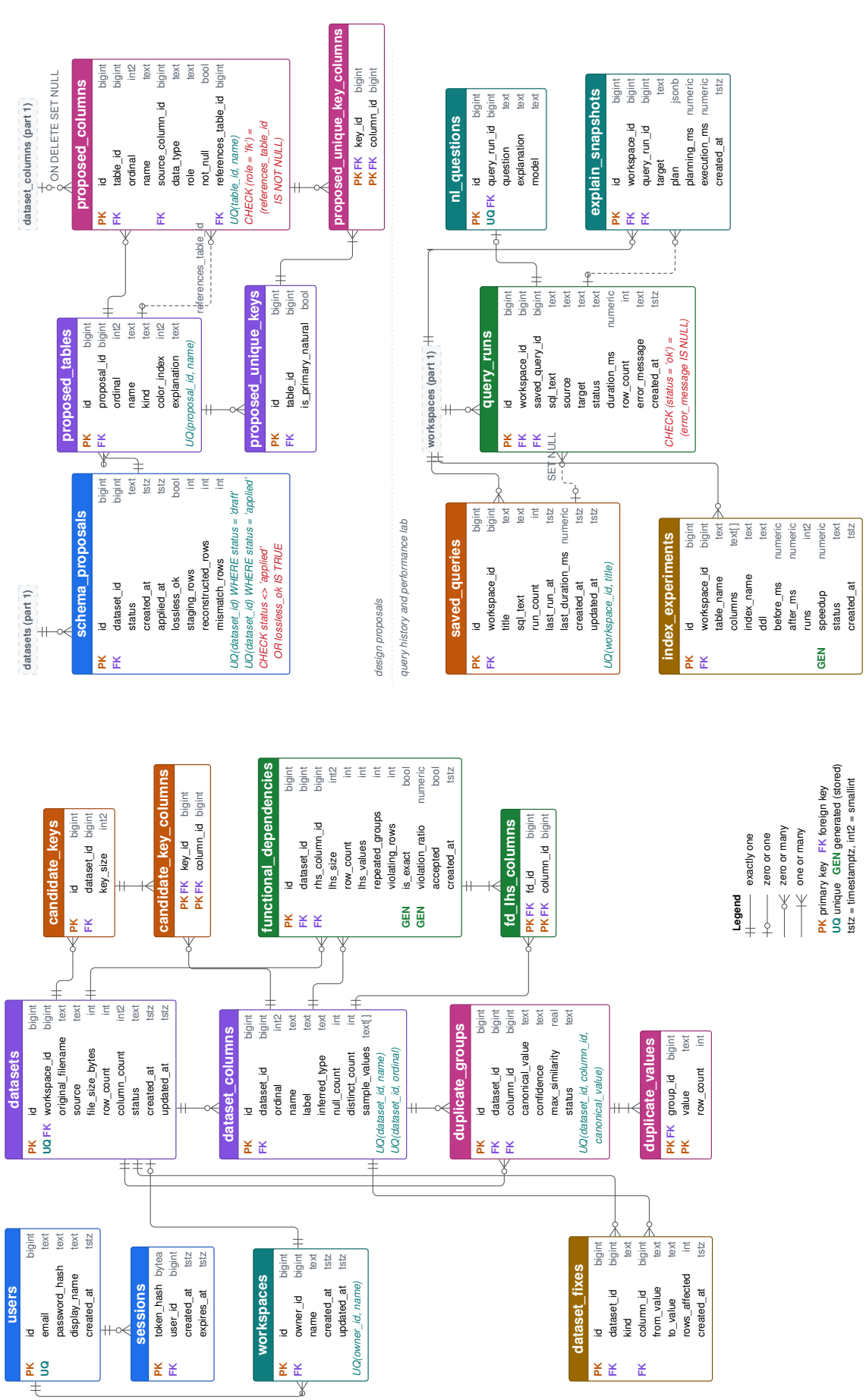


Figure 3.2: Entity–relationship diagram of the `qlens` metadata schema (22 tables) in crow's-foot notation. Left: accounts, workspaces, the dataset and the analysis results. Right: proposed designs, query history and the performance lab; grey boxes repeat tables from the left, dashed relationships are optional (nullable foreign keys).

```
    UPDATE qlens.saved_queries
       SET run_count = run_count + 1, last_run_at = NEW.created_at,
           last_duration_ms = NEW.duration_ms
     WHERE id = NEW.saved_query_id;
    RETURN NULL;
END $$;
CREATE TRIGGER query_runs_record AFTER INSERT ON qlens.query_runs
    FOR EACH ROW WHEN (NEW.saved_query_id IS NOT NULL AND NEW.status = 'ok')
    EXECUTE FUNCTION qlens.record_query_run();
```

3.3.2 Dependency Discovery

Figure 3.3 shows the pipeline. Import copies every cell into `_raw` with `COPY`, infers each column's type with one aggregate query of regular-expression tests (dates through the PL/pgSQL function `try_date`, which returns NULL instead of raising), and builds `_staging` with `CREATE TABLE ... AS SELECT` and the casts.

Discovery then asks PostgreSQL. `qlens.check_fd` (Listing 3.2) groups the staged rows by the left-hand side and the right-hand side and returns, for $X \to A$, the number of rows that would have to change for the dependency to hold: per X-group, every row except those with the most frequent A. This is the g_3 error measure of approximate dependencies [11]. A dependency is *exact* at 0 and *approximate* up to 2% of the rows, and it must be supported by at least two repeated X-groups. Identifiers enter only through `format('%I')`, so the dynamic SQL cannot be injected. Candidate keys use `qlens.is_unique` on combinations of 1–3 columns, pruned first by the profile: a key must have no NULLs, and the product of its columns' distinct counts must reach the row count.

Listing 3.2: `qlens.check_fd`: measuring a dependency with dynamic SQL (`004_analysis.sql`)

```
CREATE FUNCTION qlens.check_fd(p_schema text, p_table text, p_lhs text[], p_rhs text)
RETURNS TABLE (row_count bigint, lhs_values bigint, repeated_groups bigint, violating_rows
    bigint)
LANGUAGE plpgsql STABLE AS $$
DECLARE
    t   text := format('%I.%I', p_schema, p_table);
    lhs text;
BEGIN
    SELECT string_agg(format('%I', c), ', ') INTO lhs FROM unnest(p_lhs) AS c;
    RETURN QUERY EXECUTE format(
        'WITH c AS (SELECT %1$s, %2$I, count(*) AS cnt FROM %3$s GROUP BY %1$s, %2$I), '
        '     g AS (SELECT %1$s, sum(cnt) AS n, max(cnt) AS keep FROM c GROUP BY %1$s) '
        'SELECT (SELECT count(*) FROM %3$s), count(*), count(*) FILTER (WHERE n >= 2), '
        '       coalesce(sum(n - keep), 0)::bigint FROM g',
        lhs, p_rhs, t);
END;
$$;
```

Raw discovery is noisy, so three rules keep the list readable: measures (amounts, quantities) and flags are never determinants; a pair $XY \to A$ is dropped when X alone already (nearly) determines A; and an approximate $X \to A$ explained by an exact $X \to Z$ and $Z \to A$ is

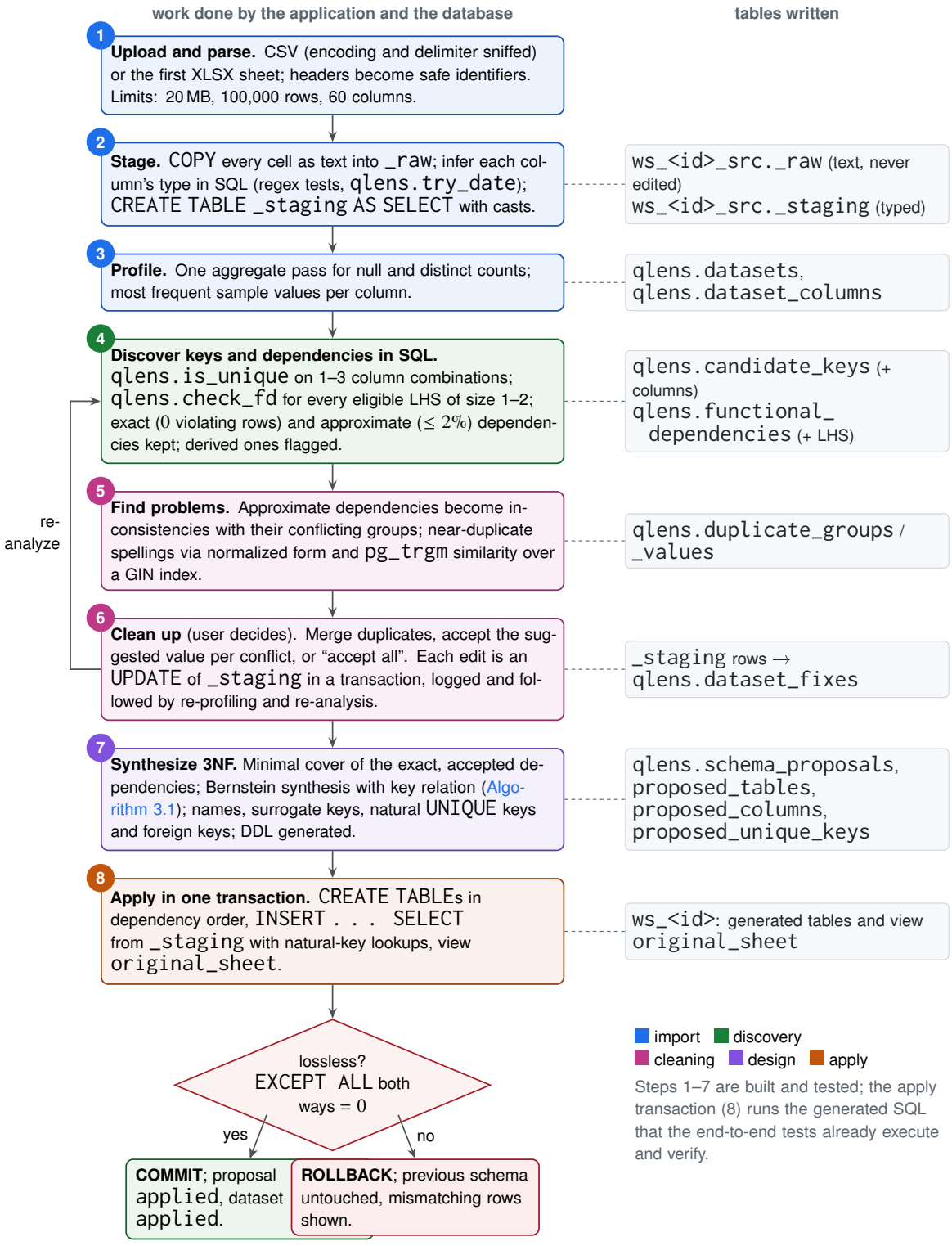


Figure 3.3: The normalization pipeline from upload to the lossless check. Left: the steps in order (colour marks the phase); right: the tables each step writes. Steps 4–6 can loop: every clean-up re-profiles the edited columns and re-runs discovery.

reported only once, at its root. Dependencies implied by the others are flagged *derived*. On the fee register this leaves exactly six core dependencies, two of them approximate: 8 rows for {`student_phone`} → `student_name` and 5 for {`teacher`} → `teacher_phone`.

3.3.3 Data Cleaning

Near-duplicates. Spellings equal after lower-casing and collapsing spaces and full stops are certain duplicates. Probable typos are found with `pg_trgm` trigram similarity [12]: candidate values go into a temporary table with a GIN `gin_trgm_ops` index, so the `%` operator finds look-alikes without comparing every pair. Two rules remove false alarms: values whose digits differ are different things (*Foundation 9* and *Foundation 10*), and a typo must be at most a third as frequent as the spelling it resembles. Matches are clustered with union–find.

Inconsistencies and repairs. Each approximate dependency lists its conflicting groups with the most frequent value as the suggestion, which is the minimal repair under g_3. A merge or fix is one `UPDATE` of `_staging` inside a transaction (`FOR UPDATE` on the affected rows), logged in `dataset_fixes` and followed by re-profiling; "accept all" merges every certain duplicate, fixes every approximate dependency and re-analyzes, so both fee-register inconsistencies become exact.

3.3.4 3NF Synthesis

The design is built from exactly the dependencies the user sees: exact, accepted and not derived. Synthesis is set algebra over column names, implemented in Python and summarized in Algorithm 3.1. The closure X^+ answers every question it needs: $X \to A$ follows from F iff $A \in X^+$ [13]. The minimal cover splits right-hand sides, removes extraneous left-hand attributes and redundant dependencies, with all tie-breaks in sheet order so the result is deterministic. Groups with equivalent determinants are merged, and Bernstein's step 5 then prunes dependencies made redundant by the equivalences [3].

Plain synthesis preserves dependencies but is lossless only if some relation contains a key of the whole table [14]. QLens therefore designates as *fact* relation the relation that contains every undetermined attribute and whose closure is the whole table, or else adds one built from a key of the dependencies. Joining outward from the fact relation, each join is on a key of the relation being joined, so it adds determined attributes and never adds or removes rows. 900 randomized cases are checked against a chase-based lossless-join test [15] and a brute-force 3NF test. Figure 3.4 shows the result for the fee register.

3.3.5 Proposal and Apply

From relations to tables. Built Every table gets `id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY`; its determinant and equivalent determinants become `UNIQUE NULLS NOT DISTINCT` constraints, so an empty *Teacher* cell is one teacher stored once. When a relation contains

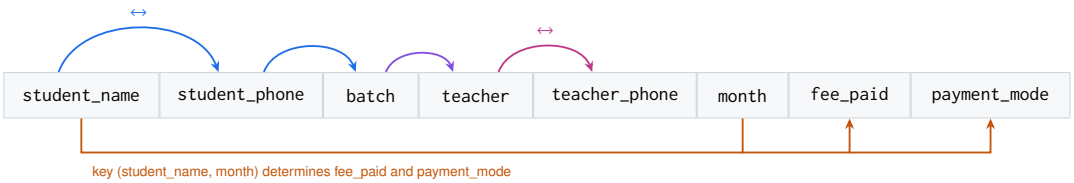
Algorithm 3.1. Lossless 3NF synthesis used by QLens (Bernstein synthesis with equivalence merging, the step-5 redundancy pruning and a key relation)

Input: attributes U in sheet order; exact, accepted dependencies F; candidate keys $K_1, \ldots, K_m$ found in the data

Output: relations $R_1, \ldots, R_n$ (entities) and one fact relation R_f; every R_i is in 3NF and $\bowtie_i R_i$ is lossless

$G \leftarrow \textsc{MinimalCover}(F)$ `// singleton RHS, left-reduced, no redundant FD`
partition G into groups $G_X = \{X \to A \in G\}$, one per left-hand side X
foreach *pair of groups* G_X, G_Y **do**
  if $X \subseteq Y_G^+$ ***and*** $Y \subseteq X_G^+$ **then** `// equivalent determinants`
    merge G_X and G_Y into one group with determinants $\{X, Y\}$
$J \leftarrow \{X \to B \mid X, Y$ determinants of one merged group, $B \in Y\}$
for $X \to A \in G$ ***from last to first*** **do** `// Bernstein step 5`
  if $A \in X^+_{(G \setminus \{X \to A\}) \cup J}$ **then** remove $X \to A$ from G
foreach *merged group with determinants* $X_1, \ldots, X_k$ **do**
  $R \leftarrow X_1 \cup \cdots \cup X_k \cup \{A \mid X_j \to A \in G\}$; key $\leftarrow$ earliest X_j in sheet order
  emit entity relation R unless it carries no dependency
$D \leftarrow$ attributes appearing on the right of some emitted relation's non-key part
if *some emitted* R *contains* $U \setminus D$ ***and*** $R_G^+ = U$ **then**
  mark R as the fact relation R_f
else
  $Z \leftarrow U$
  for $a \in U$ ***in reverse sheet order*** **do** `// FD-key by reverse reduction`
    if $(Z \setminus \{a\})_G^+ = U$ **then** $Z \leftarrow Z \setminus \{a\}$
  $R_f \leftarrow Z$; its unique key is K_1 if $K_1 \subseteq Z$, else none (surrogate key only)
drop every entity relation whose attributes are a subset of another relation's
return *entity relations, then* R_f

(a) Dependencies discovered in the 3,000-row sheet



Before cleaning: {student_phone} → student_name fails on 8 rows (variant spellings such as riya sharma) and {teacher} → teacher_phone fails on 5 rows (Mr. Gupta's old number). {student_name} → batch and {student_phone} → teacher are derived and not shown.

(b) Synthesized 3NF schema (lane colours as in the application)

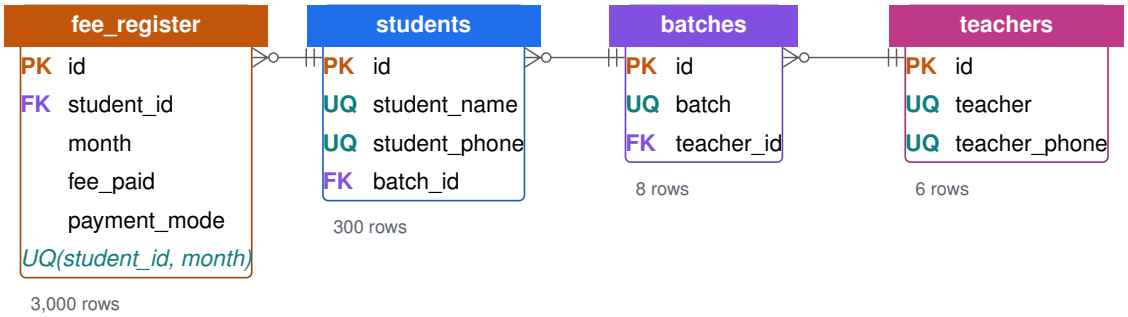


Figure 3.4: Worked example on the bundled fee register. (a) The columns of the sheet with the dependencies QLens finds; arrows point from determinant to dependent, ↔ marks a pair of mutually determining columns. (b) The schema produced by the synthesis after the 13 faulty rows are cleaned: each dependency moves to the table of its determinant, so a teacher's phone number is stored once instead of on every row. Row counts follow from the sample's construction (300 students, 8 batches, 6 teachers).

another entity's natural key, those columns are replaced by a foreign key `<entity>_id ... ON DELETE RESTRICT`, `NOT NULL` exactly when the replaced columns had no empty cells. Tables are named from their determinant (`student_name` → `students`), carry a plain-English explanation and can be renamed while the proposal is a draft (otherwise 409 `proposal_not_draft`). The SQL is generated in topological order of the foreign keys: entity loads are `INSERT ... SELECT DISTINCT` from `_staging`, foreign keys are looked up by natural key (plain `=` where the key has no NULLs, so a hash join is possible, `IS NOT DISTINCT FROM` otherwise), the fact keeps every row in sheet order, and the view `original_sheet` joins everything back into the original columns.

One transaction. Built Apply locks the draft with `SELECT ... FOR UPDATE`, drops and recreates the workspace schema, runs the generated statements and finally the check of Listing 3.3. A non-zero count raises 409 `lossless_check_failed` with up to five differing rows, and PostgreSQL rolls everything back: old tables and statuses are untouched. Otherwise the previous applied proposal is set to *superseded before* the new one becomes *applied*, since the partial unique indexes allow only one applied design per dataset, and after the commit each new table is `ANALYZE`d so that the first queries get real statistics.

Listing 3.3: The lossless check generated for the fee register (column list abbreviated as `cols`)

```
SELECT (SELECT count(*) FROM ws_20_src._staging)  AS staging_rows,
       (SELECT count(*) FROM ws_20.original_sheet) AS reconstructed_rows,
       (SELECT count(*) FROM (SELECT cols FROM ws_20.original_sheet
                              EXCEPT ALL SELECT cols FROM ws_20_src._staging) d) AS
    extra_rows,
       (SELECT count(*) FROM (SELECT cols FROM ws_20_src._staging
                              EXCEPT ALL SELECT cols FROM ws_20.original_sheet) d) AS
    missing_rows;
```

The comparison is of *multisets*: `EXCEPT` would hide a duplicated or lost copy of a repeated row, while `EXCEPT ALL` cancels rows one for one; and both directions are needed, because A `EXCEPT ALL` B is empty whenever $A \subseteq B$. On the cleaned fee register all 3,000 rows come back exactly, with 6 teachers, 8 batches and 300 students. Tests also cover forced failures, reserved-word headers (*Order*, *Group*, *User's Name*), empty determinants, the other two samples and randomly generated sheets (a 400-seed run passed throughout).

3.3.6 Query Sandbox

In progress A user query never runs on the application's pooled connection (Figure 3.5). After the workspace is checked to belong to the session's user and the text is parsed with pglast (one `SELECT`, no forbidden functions), it runs on a separate login as `qlens_ws_<id>` in a read-only transaction with a statement timeout, and at most 1,000 rows are returned. The outcome is then recorded in `query_runs` on the pooled connection, where the trigger of Listing 3.1 updates the saved query. The role, its settings, its grants, `query_runs` and the trigger are built and tested; the runner and its editor are being completed.

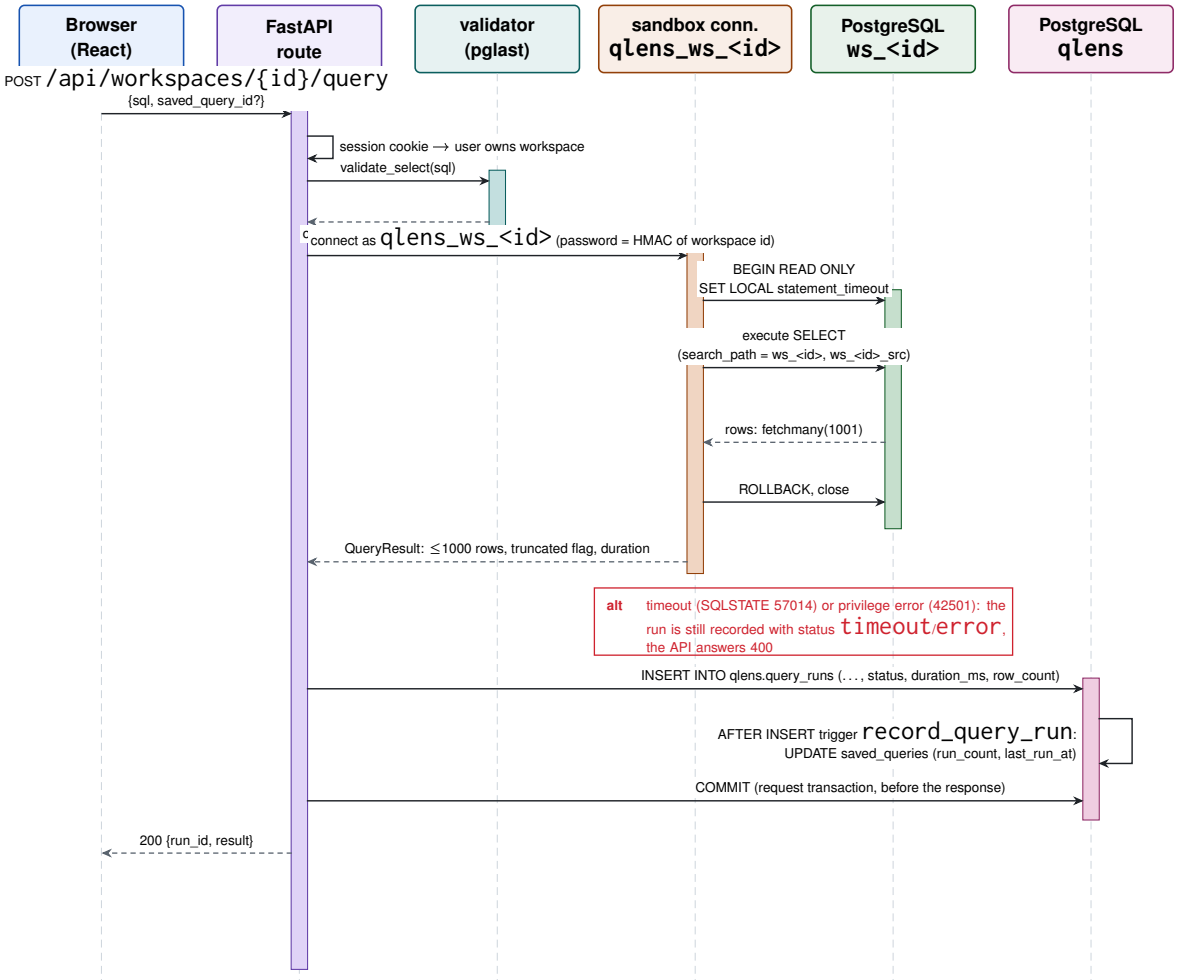


Figure 3.5: Sequence of a user query through the sandbox. The query never runs on the application's pooled connection: it runs on a separate login as the workspace role, whose privileges reach only that workspace's schemas, inside a read-only transaction with a statement timeout and a row cap. The run is then recorded on the pooled connection, where a trigger maintains the saved query's statistics. (Runner in progress; the role, its settings, `query_runs` and the trigger are built and tested.)

3.3.7 Interface and Assistant

Each generated table keeps one lane colour (`proposed_tables.color_index`) across the Analyze diagram, the ER canvas, the SQL and these figures. The landing page, dashboard and Analyze screen are Built: Analyze opens with a diagram of the sheet's structure, a banner of candidate keys and a graph of dependencies, solid when exact and dashed when approximate (Figure 3.6). Motion explains rather than decorates: diagrams enter in stages, only `transform`, `opacity` and `stroke-dashoffset` are animated, and everything is static under `prefers-reduced-motion`.

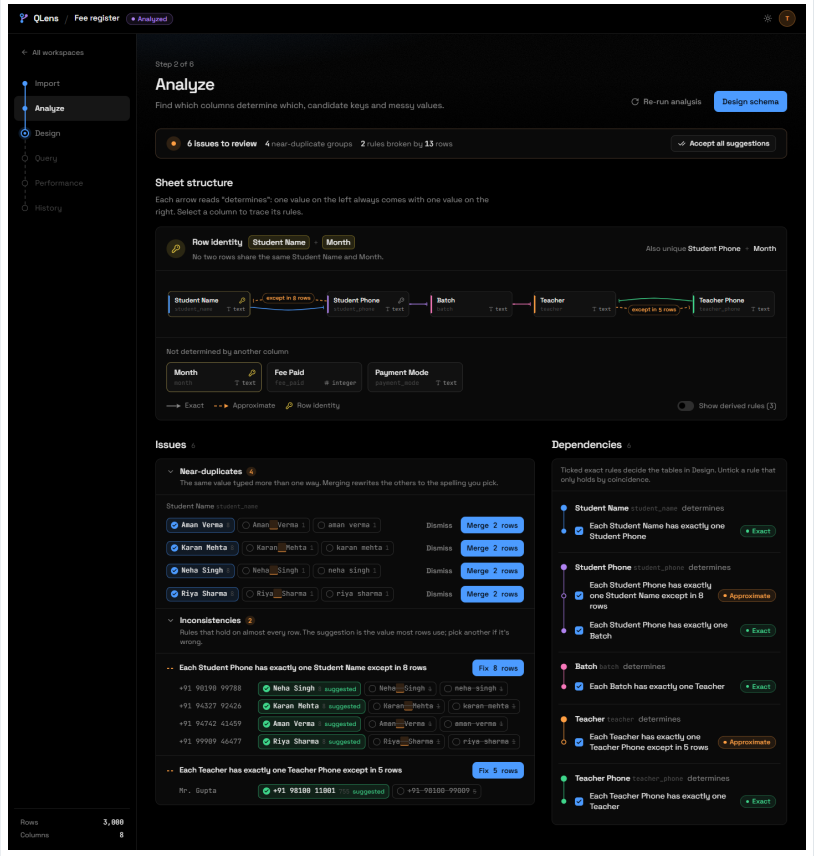


Figure 3.6: Analyze: candidate keys, the dependency graph (dashed edges are approximate) and the issues with their suggested fixes.

In progress The Design step is a dot-grid canvas with draggable, lane-coloured tables, inline renaming, a "why this design" panel, the SQL and the Apply button; its backend is complete. On it floats the command bar of the optional Gemini assistant. The model chooses, by function calling, between answering a question and reworking the draft, but it can act only through server-validated tools: `describe_schema`, `check_dependency` (which runs `qlens.check_fd`), `run_sql` (through the sandbox), `explain_query`, `rename_table`/`rename_column` (same checks as the user's renames), `set_dependency_accepted` (re-synthesizes) and `move_column`, allowed only when `check_fd` shows the target's determinant determines the column exactly. Every edit creates a new draft version, so it can be undone, and conversations and tool calls are stored in the `qlens` schema.

CHAPTER 4

Conclusion

QLens shows that a person who owns only a messy spreadsheet can be taken, step by step and with every step explained, to a verified third-normal-form database. Type inference, key search and dependency discovery run inside PostgreSQL through PL/pgSQL and dynamic SQL; near-duplicates are found with `pg_trgm` over a GIN index; repairs are transactional and audited; the synthesis is Bernstein's algorithm with a key relation, property-tested for 3NF and losslessness; and the apply step proves on the real rows, with `EXCEPT ALL` in both directions inside one transaction, that nothing was lost. On the fee register every planted problem is found and the expected four-table design rebuilds all 3,000 rows exactly. Around this sits a 22-table metadata schema that uses generated columns, `CHECK` rules, partial unique indexes, triggers and window-function views, and a per-workspace login role that isolates users by privilege. 229 backend and 63 frontend tests pass.

Lessons. Dependencies in real data are statistical before they are logical, so measuring violations (g_3) and letting the user decide was the central design decision. Normalization needs clean data first: a rule broken by eight typos would otherwise be lost and the design would keep a redundant column. Set-oriented checks belong in the database. Constraints are documentation that executes: they caught our own mistakes, such as an applied-design rule that let `NULL` pass until it was written as `IS TRUE`. And losslessness, a theorem in the textbook, becomes in practice a query.

Limitations. Left-hand sides are limited to two columns and keys to three; noise filtering relies on thresholds, so the user keeps the final say; one flat sheet of up to 100,000 rows is normalized at a time, and only to 3NF; trigram similarity cannot know that two people share a name; and the evidence comes from synthetic samples and tests, not a user study. The query editor, design canvas and assistant are In progress, and the performance laboratory and step-through visualizer are Planned.

Future work. Completing the sandboxed editor and the canvas with its validated assistant; a performance laboratory that renders `EXPLAIN (ANALYZE, BUFFERS)` as a tree and times index experiments on a scaled copy; a clause-by-clause visualizer for teaching; wider dependency search with sampling and multivalued dependencies for 4NF; and resource limits for hosted, multi-tenant use, which the per-workspace role design was chosen to make possible.

References

[1] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[2] W. Kent, "A simple guide to five normal forms in relational database theory," *Communications of the ACM*, vol. 26, no. 2, pp. 120–125, 1983.

[3] P. A. Bernstein, "Synthesizing third normal form relations from functional dependencies," *ACM Transactions on Database Systems*, vol. 1, no. 4, pp. 277–298, 1976.

[4] The PostgreSQL Global Development Group, "PostgreSQL 18 documentation," https://www.postgresql.org/docs/18/, 2025.

[5] K. Beck, *Test-Driven Development: By Example*. Addison-Wesley, 2002.

[6] Meta Open Source, "React documentation," https://react.dev/, 2025.

[7] S. Ramírez, "FastAPI documentation," https://fastapi.tiangolo.com/, 2025.

[8] D. Varrazzo, "Psycopg 3: PostgreSQL database adapter for Python," https://www.psycopg.org/psycopg3/docs/, 2025.

[9] The PostgreSQL Global Development Group, "pg_trgm: Support for similarity of text using trigram matching," PostgreSQL 18 Documentation, Appendix F, https://www.postgresql.org/docs/18/pgtrgm.html, 2025.

[10] L. Gaifax, "pglast: PostgreSQL languages AST and statements prettifier," https://pglast.readthedocs.io/, 2025.

[11] J. Kivinen and H. Mannila, "Approximate inference of functional dependencies from relations," *Theoretical Computer Science*, vol. 149, no. 1, pp. 129–149, 1995.

[12] R. C. Angell, G. E. Freund, and P. Willett, "Automatic spelling correction using a trigram similarity measure," *Information Processing & Management*, vol. 19, no. 4, pp. 255–261, 1983.

[13] W. W. Armstrong, "Dependency structures of data base relationships," in *Proceedings of the IFIP Congress*, 1974, pp. 580–583.

[14] J. Biskup, U. Dayal, and P. A. Bernstein, "Synthesizing independent database schemas," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1979, pp. 143–151.

[15] A. V. Aho, C. Beeri, and J. D. Ullman, "The theory of joins in relational databases," *ACM Transactions on Database Systems*, vol. 4, no. 3, pp. 297–314, 1979.

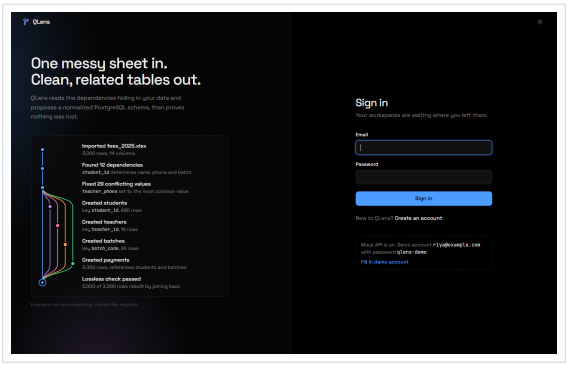
CHAPTER A

Sample Datasets and Screens

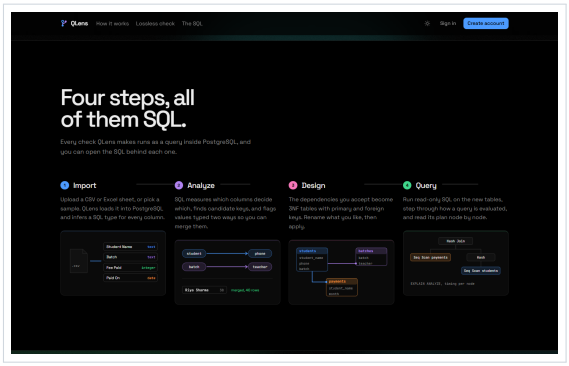
The three bundled sheets are generated deterministically (seed 2026), and their planted problems are asserted by the tests, so every demonstration sees identical numbers.

Table A.1: Bundled sample sheets and their planted problems.

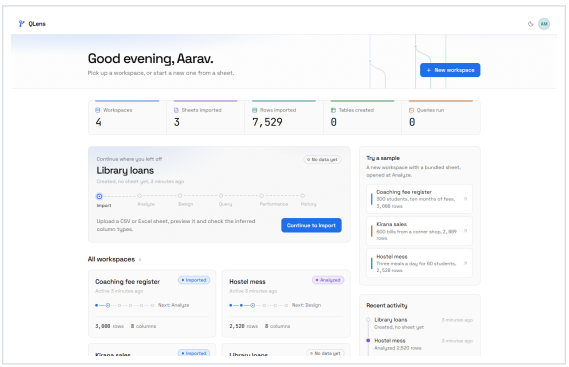
Sheet	Rows	Structure	Planted problems
Coaching fee register	3,000	300 students × 10 months; 8 batches, 6 teachers; key (student, month); about 8% unpaid and 15% partial payments	4 names in variant spellings (8 rows); Mr. Gupta's old phone (5 rows); a genuine look-alike, Priya Sharma, must not be merged
Kirana sales	2,009	600 bills, 80 customers, 40 items in 6 categories; key (bill, item); amount = price × qty	old price of Toor Dal (3 rows); *Mohan lal* for *Mohan Lal* (3 rows)
Hostel mess	2,520	60 students × 14 days × 3 meals; room → block → warden; one menu item per (date, meal)	*Mrs Rao* for *Mrs. Rao* (2 rows)



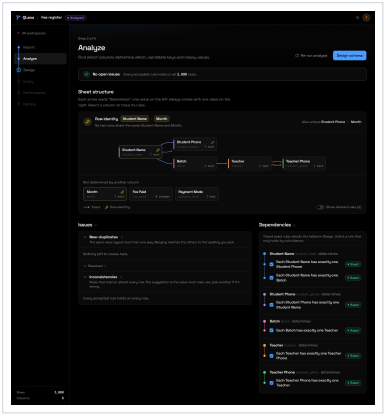
(a) Sign-in



(b) Landing: how it works



(c) Dashboard, light theme



(d) Analyze after clean-up

Figure A.1: Further screens of QLens.